Capítulo 2 - Conceitos e arquitetura do sistema de banco de dados

💡 ATENÇÃO: O CAPITULO 2 NÃO FOI ESTUDADO EM SUA TOTALIDADE DENTRO DESTA MATÉRIA. DESSA FORMA ESSE RESUMO VAI ATÉ A PARTE QUE O PROFESSOR PASSOU E NÃO CONTÉM AS ATIVIDADES DO CAPITULO

Conceitos e arquitetura do sistema de banco de dados

Modelos de dados, esquemas e instâncias

Uma característica fundamental da abordagem de banco de dados é que ela oferece algum nível de abstração. A **abstração de dados** geralmente se refere à supressão de detalhes de organização e armazenamento, destacando os recursos essenciais para melhor conhecimento desses dados. Uma das principais características da abordagem de banco de dados é possibilitar a abstração , de modo que diferentes usuários possam percebê-los em seu nível de detalhe preferido. Um **modelo de dados** - uma coleção de conceitos que podem ser usados para descrever a estrutura de um banco de dados - oferece os meios necessários para alcançar essa abstração. Com *estrutura de um banco de dados*, queremos dizer os tipos, os relacionamentos e as restrições que se aplicam aos dados. A maioria dos modelos de dados também inclui um conjunto de **operações básicas** para especificar recuperações e atualizações no banco de dados. Além das operações básicas fornecidas pelo modelo de dados, está se tornando mais comum incluir conceitos no modelo de dados para especificar o **aspecto dinâmico ou comportamento** de uma aplicação de banco de dados. Isso permite ao projetista especificar um conjunto de operações válidas, definidas pelo usuário, sobre os objetos do banco de dados. Um exemplo de uma operação definida pelo usuário poderia ser CALCULA_MEDIA, que pode ser aplicada a um objeto ALUNO. Por sua vez, operações genéricas para inseris, excluir modificar ou recuperar qualquer tipo de objeto normalmente estão incluídas nas operações básicas do modelo de dados. Conceitos para especificar o comportamento são

fundamentais para os modelos de dados orientados a objetos, mas também estão sendo incorporados em modelos de dados tradicionais.

Categorias de modelos de dados

- **Modelos de dados de alto nível ou conceituais** oferecem conceitos próximos ao modo como muitos usuários percebem os dados;
- **Modelos de dados de baixo nível ou físicos** oferecem conceitos que descrevem os detalhes de como os dados são armazenados no computador, em geral em discos magnéticos.
- Entre esses dois extremos está uma classe de **modelos de dados representativos** (ou de **implementação**), que oferece conceitos que podem ser facilmente entendidos pelos usuários finais, mas que não estão muito longe do modo como os dados são organizados e armazenados no computador.
- Uma **entidade** representa um objeto ou conceito do mundo real, como um funcionário ou um projeto de minimundo que é descrito no banco de dados.
- Um **atributo** representa alguma propriedade de interesse que descreve melhor uma entidade, como nome ou salário do funcionário.
- Um **relacionamento** entre duas entidades representa uma associação entre elas, por exemplo um relacionamento trabalha-em, entre um funcionário e um projeto.
- Um c**aminho de acesso** é um estrutura que torna eficiente a busca por registros de um banco de dados em particular, como indexação ou hashing.
- Um **índice** é um exemplo de caminho de acesso que permite o acesso direto aos dados usando um termo de índice ou uma palavra-chave. Pode ser organizado de forma linear, hierárquica ou de alguma outra maneira.
- Outra classe de modelo de dados é conhecida como **modelo de dados auto descritivos**. O armazenamento de dados nos sistemas baseados nesses modelos combina a descirção dos dados com seus próprios valores. Nos SGBD tradicionais, a descrição(esquema) fica separada dos dados. Esses modelos incluem XML, além de muitos **armazenamentos de chave-valor e sistemas NOSQL**, que foram criados recentemente para gerenciar big data.

Esquemas, instâncias e estados do banco de dados

- A descrição de um banco de dados é chamada de **esquema do banco de dados**, que é especificado durante o projeto e não se espera que mude com frequência.

- A representação de um esquema é chamada de diagrama de esquemas.
- Chamamos cada objeto no esquema - por exemplo, ALUNO ou DISCIPLINA - de **construtor do esquema**.
- Os dados reais armazenados em um banco de dados podem mudar com bastante frequência.
- Os dados no banco de dados em determinado momento do tempo são chamados de **estado ou instante (snapshot) do banco de dados**. Também são chamados de conjunto atual de **ocorrências ou instâncias** no banco de dados.

Quando definimos um novo banco de dados, especificamos seu esquema apenas para o SGBD. Nesse ponto, o estado correspondente é o estado vazio, sem dados. Obtemos o estado inicial do banco de dados quando ele é populado ou carregado com os dados iniciais. Daí em diante, toda vez que uma operação de atualização é aplicada ao banco de dados, obtemos outro estado. Em qualquer ponto no tempo, o banco de dados tem um estados atual. O SGBD é parcialmente responsável por garantir que todo estado do banco de dados seja um estado válido - ou seja, um estado que satisfaça a estrutura e as restrições especificadas no esquema. Logo, especificar um esquemas correto para o SGBD é extremamente importante, e o esquema deve ser projetado com extremo cuidado. O SGBD armazena as descrições das construções e restrições do esquema - também denominadas **metadados** - no catálogo do SGBD, de modo que o software do SGBD possa recorre ao esquema sempre que precisar. O esquema às vezes é chamado de **intenção**, e um estado do banco de dados é chamado de **extensão**.

Embora, como já mencionado, o esquema não deva mudar com frequência, não é raro que as mudanças ocasionalmente precisem ser aplicadas ao esquema, à medida que os requisitos da aplicação mudam. Isso é conhecido como **evolução do esquema**.

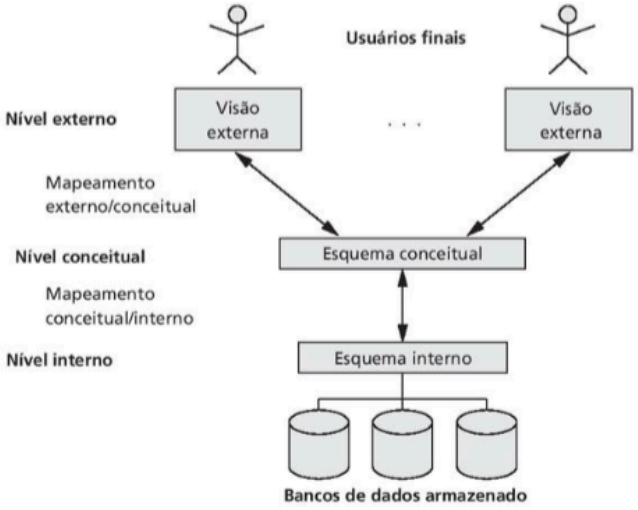
Arquitetura de três esquemas e independência de dados

Três das quatro características importantes da abordagem de banco de dados são, (1) uso de um catálogo para armazenar a descrição (esquema) do banco de dados de modo a torná-lo auto descritivo, (2) isolamento de programas e de dados (independência entre dados e operação do programa) e (3) suporte para múltiplas visões do usuário.

A arquitetura de três esquemas

O objetivo da arquitetura de três esquemas é separar as aplicações do usuário do banco de dados físico. Nessa arquitetura, os esquemas podem ser definidos nos três níveis a seguir:

1. O nível interno tem um esquema interno, que descreve a estrutura do armazenamento físico do banco de dados. O esquema interno usa um modelo de dados físico e descreve os detalhes completos do armazenamento de dados e os caminhos de acesso para o banco de dados.
2. O nível conceitual tem um esquema conceitual, que descreve a estrutura do banco de dados inteiro para uma comunidade de usuários. O esquema conceitual oculta os detalhes das estruturas de armazenamento físico e se concentra na descrição de entidades, tipos de dados, relacionamentos, operações do usuário e restrições. Normalmente, um modelo de dados representativo é usado para descrever o esquema conceitual quando um sistema de banco de dados é implementado. Esse esquema conceitual de implementação costuma ser baseado com um projeto de esquema conceitual em um modelo de dados de alto nível.
3. O nível externo ou de visão inclui uma série de esquemas externos ou visões do usuário. Cada esquema externo descreve a parte do banco de dados em que um grupo de usuários em particular está interessado e oculta o restante do banco de dados do grupo de usuários. Como no nível anterior, cada esquema externo é comumente implementado com o uso de um modelo de dados representativo, possivelmente fundamentado em um projeto de esquema externo em um modelo de dados de alto nível.



Observe que os três esquemas são apenas descrições dos dados; os dados armazenado que realmente existem estão apenas no nível físico. Na arquitetura de três esquemas, cada grupo de usuário recorre ao seu próprio esquema externo. Assim, o SGBD precisa transformar uma solicitação especificada em um esquema externo em uma solicitação no esquema conceitual, e depois em uma solicitação no esquema interno para o processamento no banco de dados armazenado. Se a solicitação for uma recuperação no banco de dados, os dados extraídos do banco de dados armazenados devem ser reformatados para corresponder a visão externa do usuário. Os processos de transformação de requisições e os resultados entre os níveis são chamados de **mapeamentos**. Esses mapeamentos podem ser demorados, de modo que alguns SGBD - especialmente aqueles que servem para dar suporte a pequenos bancos de dados - não oferecem suporte a visão externas. Porém, mesmo em tais sistemas, é necessário transformar solicitações entre os níveis conceitual e interno.

Independência de dados

Independência de dados pode ser definida como a capacidade de alterar o esquema em um nível do sistema de banco de dados sem ter que alterar o esquema no próximo nível mais lato. Podemos definir dois tipos de independência de dados:

1. **Independência lógica de dados** é a capacidade de alterar o esquema conceitual sem ter de alterar os esquemas externos ou os programas de aplicação. Podemos alterar o esquema conceitual para expandir o banco de dados (acrescentando um tipo de registro ou item de dado), para alterar restrições ou para reduzi-lo (removendo um tipo de registro ou item de dado). No último caso, esquemas externos que ser referem apenas aos dados restante não seriam afetados. Depois que o esquema conceitual passa por uma reorganização lógica, os programas de aplicação que referenciam as construções do esquema externo devem funcionar da mesma forma que antes. As mudanças nas restrições podem ser aplicadas ao esquema conceitual sem afetar os esquemas externos ou programas de aplicação.

2. Independência física de dados é a capacidade de alterar o esquema interno sem ter de alterar o esquema conceitual. Logo, os esquemas externos também não precisam ser alterados. Mudanças no esquema interno pode ser necessárias porque alguns arquivos físicos foram reorganizados - por exemplo, ao criar estruturas de acesso adicionais - para melhorar o desempenho de recuperação ou atualização. Se os mesmo dados de antes permanecem no banco de dados, provavelmente não teremos de alterar o esquema conceitual.

Em geral, a independência física de dados existe na maioria dos bancos de dados e ambientes de arquivo, nos quais detalhes físicos, como a localização exata dos dados no disco, e detalhes de hardware sobre codificação do armazenamento. posicionamento, compactação, divisão, fusão de registros, e assim por diante são ocultados do usuário. As demais aplicações desconhecem esses detalhes. Por sua vez, a independência lógica de dados é mais difícil de ser alcançada porque permite alterações estruturais e de restrição sem afetar programas de aplicação - um requisito muito mais estrito.

Sempre que temos um SGBD de múltiplos níveis, seu catálogo deve ser expandido para incluir informações sobre como mapear solicitações e dados entre os diversos níveis. O SGBD usa software adicional para realizar esses mapeamentos, recorrendo à informação de mapeamento no catálogo. A independência de dados ocorre porque, quando o esquema é alterado em algum nível, o esquema no próximo nível mais alto permanece inalterado; somente o mapeamento entre dois níveis é alterado. Portanto, os programas de aplicação que fazem referência ao esquema de nível mais alto não precisam ser alterados.

Linguagens e interfaces de banco de dados

Linguagens do SGBD

Quando o projeto do banco de dados é finalizado e um SGBD é escolhido para implementá-lo, o primeiro passo é especificar esquemas conceituais e internos para o banco de dados e quaisquer mapeamentos entre os dois. Em muitos SGBD, em que não é mantida nenhuma separação estrita de níveis, uma linguagem, chamada linguagem de definição de dados (DDL), é usada pelo DBA e pelos projetistas de banco de dados para definir os dois esquemas. O SGBD terá um compilador da DDL cuja função é processar instruções da DDL a fim de identificar as descrições dos construtores de esquema e armazenar a descrição de esquema no catálogo do SGBD.

Nos SGBD que mantêm uma separação clara entre os níveis conceitual e interno, a DDL é usada para especificar apenas o esquema conceitual. Outra linguagem, a linguagem de definição de armazenamento(SDL), é utilizada para especificar o esquema interno. O mapeamento entre os dois esquemas podem ser especificados em qualquer uma dessas linguagens. Na maioria dos SGBD relacionais, não existe uma linguagem específica que realize o papel de SDL. Em vez disso, o esquema interno é especificado por uma combinação de funções, parâmetros e especificações relacionadas ao armazenamento de arquivos. Estes permitem aos DBS controlar opções de indexação e mapeamento de dados que serão armazenados. Para uma

verdadeira arquitetura de três esquemas, precisaríamos de uma terceira linguagem, a linguagem de definição de visão (VDL), para especificar visões de usuários e seus mapeamentos ao esquema conceitual, mas na maioria dos SGBD a DDL é usadas para definir tanto o esquema conceitual quanto o externo. Nos SGBD, a SQL é usadas no papel de VDL para definir visões do usuário ou da aplicação como resultado de consultas predefinidas.

Quando os esquemas do banco de dados são compilados e o banco é populado com dados, os usuários precisam de alguma forma para manipulá-lo. As manipulações típicas incluem recuperação, inserção exclusão e modificação dos dados. O SGBD oferece um conjunto de operações ou uma linguagem chamada linguagem de manipulação de dados (DML) para essas finalidades.

Nos SGBD atuais, esses tipos de linguagens normalmente não são consideradas linguagens distintas; ao contrário, uma linguagem integrada e abrangente é usadas na definição do esquema conceitual, definição de visão e manipulação de dados. A definição do armazenamento, em geral, é mantida em separado, pois serve para definir estruturas de armazenamento físicas, para ajustar desempenho do sistema de banco de dados, o que é normalmente feito pela esquipe de DBA. Um exemplo típico de linguagem de banco de dados abrangente é a linguagem relacional SQL, que representa uma combinação de DDL, VDL e DML, bem como as instruções para especificação de restrição, evolução de esquema e outros recursos. A SDL era um componente nas primeiras versões da SQL, mas foi removida da linguagem para mantê-la apenas nos níveis conceitual e externo.

Existem dois tipos de DML, Uma DML de alto nível ou não procedural pode ser utilizada para especificar operações de banco de dados complexas de forma concisa. Muitos SGBD permitem que instruções DML de alto nível sejam inseridas interativamente em um monitor ou terminal ou sejam embutidas em uma linguagem de programação de uso geral. Nesse último caso, as instruções DML precisam ser identificadas dentro do programa, de modo que possam ser extraídas por um pré-compilador e processadas por um SGBD. Uma DML de baixo nível ou procedural deve ser embutida em uma linguagem de programação de uso geral. Esse tipo de DML recupera registros individuais ou objetos do banco de dados e processa cada um deles separadamente. Portanto, precisa de construções de linguagem de programação, como laços, para recuperar e processar cada registro de um conjunto de registros. DML de baixo nível também são chamadas de DML que tratam um registro de cada vez, em virtude dessa propriedade. AS DML de alto nível como a SQL, podem especificar e recuperar muitos registros de uma única instrução DML; portanto, elas são DML chamadas de DML de um conjunto de cada vez ou orientada a

conjunto. Uma consulta em uma DML de alto nível normalmente especifica quais dados recuperar, em vez de como recuperá-los; portanto, essas linguagens também são chamadas declarativas.